

Tema 6: Estructuras de Datos Estáticas

(Vectores y Matrices)

MOMENTO 1: PRÁCTICA

EL LÍMITE DE LAS VARIABLES Y EL ARCHIVERO DIGITAL

Hasta este momento, hemos trabajado con variables simples. Sabemos que una variable es como una pequeña caja en la memoria RAM de la computadora donde podemos guardar un solo dato a la vez. Si guardamos el número 5 en la variable **notaFinal** y luego guardamos un 8, el 5 se borra para siempre y es reemplazado por el 8.

Ahora, imaginemos una situación real en la administración académica del Centro de Educación Alternativa. Como técnico encargado de sistemas, se te solicita desarrollar un programa que almacene las calificaciones finales de los 45 estudiantes del módulo de Ofimática para calcular el promedio del curso y buscar la nota más alta.

Si utilizáramos lo que sabemos hasta ahora, tendríamos que declarar 45 cajas separadas:

```
entero notaEstudiante1 = 65;  
entero notaEstudiante2 = 80;  
entero notaEstudiante3 = 51;  
// ... y así sucesivamente hasta llegar a notaEstudiante45
```

Esta forma de programar no solo es extenuante e impráctica, sino que nos impediría usar un Bucle **Para (For)** para automatizar cálculos, ya que cada variable tiene un nombre completamente distinto. Necesitamos una forma de agrupar todos esos datos bajo un mismo nombre, organizándolos en compartimentos numerados.

La Analogía del Casillero Institucional

Piensa en un mueble casillero (locker) en los pasillos del CEA. El mueble entero tiene un solo nombre, por ejemplo: "Casillero de Profesores". Sin embargo, ese único mueble está dividido en 20 compartimentos pequeños. Para dejarle un documento a un profesor, no nombras la caja por separado; dices: "Déjalo en el Casillero de Profesores, en el **compartimento número 5**".

En informática, este mueble subdividido es exactamente lo que llamamos un **Arreglo (Array) o Vector**.

MOMENTO 2: TEORÍA

VECTORES Y MATRICES: ORGANIZACIÓN DE LA MEMORIA RAM

Las Estructuras de Datos Estáticas son espacios de memoria contiguos que permiten almacenar una colección de elementos, siempre y cuando todos los elementos sean exactamente del **mismo tipo de dato** (todos enteros, todos decimales, o todos texto).

1. Arreglos Unidimensionales (Vectores)

Un Vector es una lista en línea recta (una sola dimensión). Se caracteriza por tener un nombre único, un tamaño fijo (la cantidad de "cajas" que contiene) y un sistema de numeración interna llamado **Índice**.

REGLA DE ORO INTERNACIONAL: El Índice Cero (Index 0)

En la inmensa mayoría de lenguajes de programación profesionales (C++, Java, Python, JavaScript, PHP), los procesadores **no empiezan a contar desde el 1, empiezan desde el 0**.

Si creas un vector de 5 posiciones para guardar notas, los números de los compartimentos (los índices) serán: 0, 1, 2, 3 y 4. ¡No existe la posición 5! Intentar guardar un dato en la posición 5 causará un error fatal en el sistema.

Declaración y Asignación de Valores en un Vector:

```
// 1. Declarar un vector llamado "notas" que contiene 5 espacios para números enteros
entero[] notas = nuevo entero[5];

// 2. Guardar datos en compartimentos específicos usando corchetes [ ]
notas[0] = 65; // Se guarda en el primer casillero
notas[1] = 80; // Se guarda en el segundo casillero
notas[2] = 51;
notas[3] = 90;
notas[4] = 100; // Se guarda en el quinto (y último) casillero
```

2. La Fusión Perfecta: Bucles y Vectores

La razón principal por la que creamos Vectores es para poder recorrerlos usando la estructura repetitiva **Para (For)** que aprendimos en el Tema 5. En lugar de escribir el número del compartimento a mano, usamos la variable del contador (la **i**) como si fuera una llave maestra que va abriendo caja por caja.

```
// Mostrar todas las notas de los estudiantes automáticamente
para (entero i = 0; i < 5; i++) {
    Imprimir("La nota guardada en la posición " + i + " es: " + notas[i]);
}
```

3. Arreglos Bidimensionales (Matrices)

Si un Vector es como una fila de casilleros, una Matriz es como una **hoja de cálculo de Excel** o un tablero de ajedrez. Tiene dos dimensiones: **Filas (horizontales) y Columnas (verticales)**. Para encontrar o guardar un dato en una matriz, necesitamos darle al programa dos coordenadas numéricas: [Fila][Columna].

Las matrices son fundamentales para desarrollar sistemas más complejos, como la programación de horarios de clases del CEA, donde las filas podrían ser los días de la semana y las columnas los periodos de tiempo.

```
// Declarar una matriz de 3 filas y 4 columnas
entero [][] tablero = nuevo entero [3][4];

// Guardar el número 99 en la segunda fila (índice 1) y tercera columna
(índice 2)
tablero[1][2] = 99;
```

MOMENTO 3: VALORACIÓN

IMPACTO EN LA ESTABILIDAD DE SISTEMAS Y HARDWARE

Comprender cómo se estructuran los datos en la memoria no es un capricho teórico; es la base para entender por qué los sistemas informáticos fallan en el mundo real.

El Error Crítico de Desbordamiento (Index Out of Bounds)

Cuando un desarrollador comete un error en el límite de un bucle y le ordena a la computadora buscar un dato en la posición 5 de un vector que solo tiene posiciones del 0 al 4, el procesador intenta acceder a un sector de la memoria RAM que no le pertenece. En lenguajes de alto nivel modernos, esto genera una excepción (un error en pantalla) que cierra el programa. Sin embargo, en lenguajes de programación de bajo nivel como C o C++ (usados para programar microcontroladores, sistemas operativos y sistemas biométricos), acceder a memoria prohibida puede sobrescribir datos críticos del sistema operativo, provocando un reinicio forzado del dispositivo (el temido "Pantallazo Azul") o vulnerabilidades de seguridad graves donde hackers pueden inyectar código malicioso.

Procesamiento de Imágenes y Pantallas

Cada vez que tomamos una fotografía con el celular o visualizamos un gráfico en un monitor, estamos viendo una gigantesca matriz matemática. Una pantalla con resolución Full HD (1920x1080) es literalmente una Matriz de más de 2 millones de posiciones bidimensionales. Cada celda de esa matriz guarda un código de color (píxel). Al dominar los arreglos y las matrices, estamos dando el primer paso hacia el desarrollo de gráficos, la inteligencia artificial y el análisis masivo de datos (Big Data).

MOMENTO 4: PRODUCCIÓN

CUADERNO DE TRABAJO: INGENIERÍA DE DATOS

Realiza las siguientes actividades prácticas de lógica de programación en tu cuaderno para consolidar el manejo de la memoria estática.

Actividad 1: Asignación de Memoria y Rastreo Visual

Dada la siguiente declaración de código de un Vector llamado *edades*, dibuja en tu cuaderno el arreglo completo indicando claramente los índices debajo de cada "caja" y el valor que queda guardado adentro luego de ejecutar las siguientes instrucciones:

```
entero[] edades = nuevo entero[6];

edades[0] = 18;
edades[3] = 22;
edades[1] = 19;
edades[4] = edades[0] + 2;
edades[5] = 30;
edades[2] = edades[5] - 10;
edades[0] = 25; // ;Atención a la sobrescritura!
```

Actividad 2: Traducción Lógica (Búsqueda en un Vector)

Eres el desarrollador de un sistema para recursos humanos. Tienes un vector ya lleno de datos llamado **salarios** que contiene el sueldo de 10 trabajadores. Escribe un fragmento de pseudocódigo utilizando un ciclo **Para (For)** y una condición **Si (If)** que recorra todo el vector y **cuenta cuántos trabajadores ganan más de 2500 Bs.** Guarda ese conteo en una variable acumuladora y muéstrala al final.

Actividad 3: Análisis de Errores (Depuración de Código)

Un colega intentó automatizar el llenado de un vector de 3 posiciones para guardar el número de inasistencias de los estudiantes. Ejecutó este código y su computadora colapsó mostrando un error de "Index Out of Bounds" (Índice fuera de los límites). Identifica dónde está el error y explica en tu cuaderno cómo solucionarlo.

```
entero[] inasistencias = nuevo entero[3];

para (entero i = 1; i <= 3; i++) {
    inasistencias[i] = 0;
}
```

Actividad 4: Implementación en Laboratorio (Flowgorithm) Dirígete a tu terminal de trabajo en el CEA y abre *Flowgorithm*.

1. Declara un arreglo de tipo Entero con un tamaño de 5 (recuerda que los índices irán de 0 a 4).
2. Utiliza un ciclo For para pedirle al usuario por teclado que introduzca 5 números y guárdalos en el arreglo.
3. Una vez terminado, crea un SEGUNDO ciclo For que recorra nuevamente el arreglo, sume todos los números en una variable acumuladora, y finalmente calcule e imprima el promedio exacto de los 5 números introducidos.